Embedded Data Structure
(Lecture Note 10)

Stack

2025. 4. 2

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

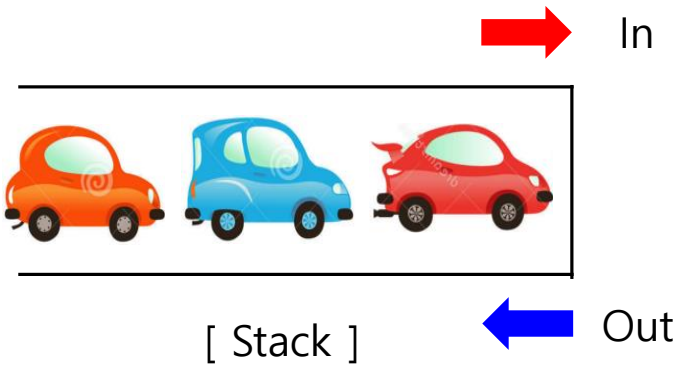
Review of Last Class

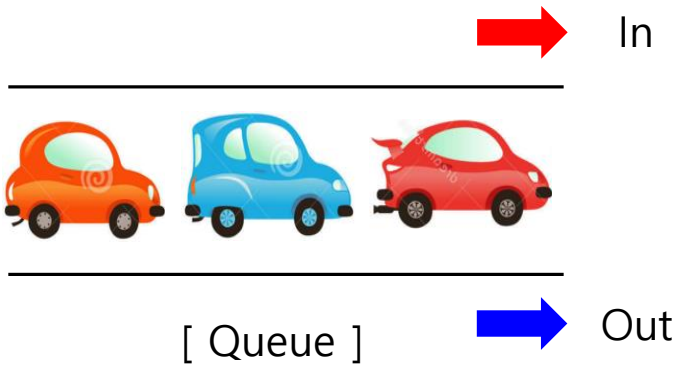
- **Simple linked list is very useful for data management**
  - **Remember the node and link structure**
  - **Namecard management : try to understand all the procedure**

    (Of course, you need to complete your implementation !)

Introduction (1/2)

- **From linked list to "stack" and "queue"**
  - **Difference by the "wall" in the parking lot !**





- Last in first out vs. first in first out
- Stack and queue : "constrained access" to make data structure
- Two types of implementation : array vs. linked list

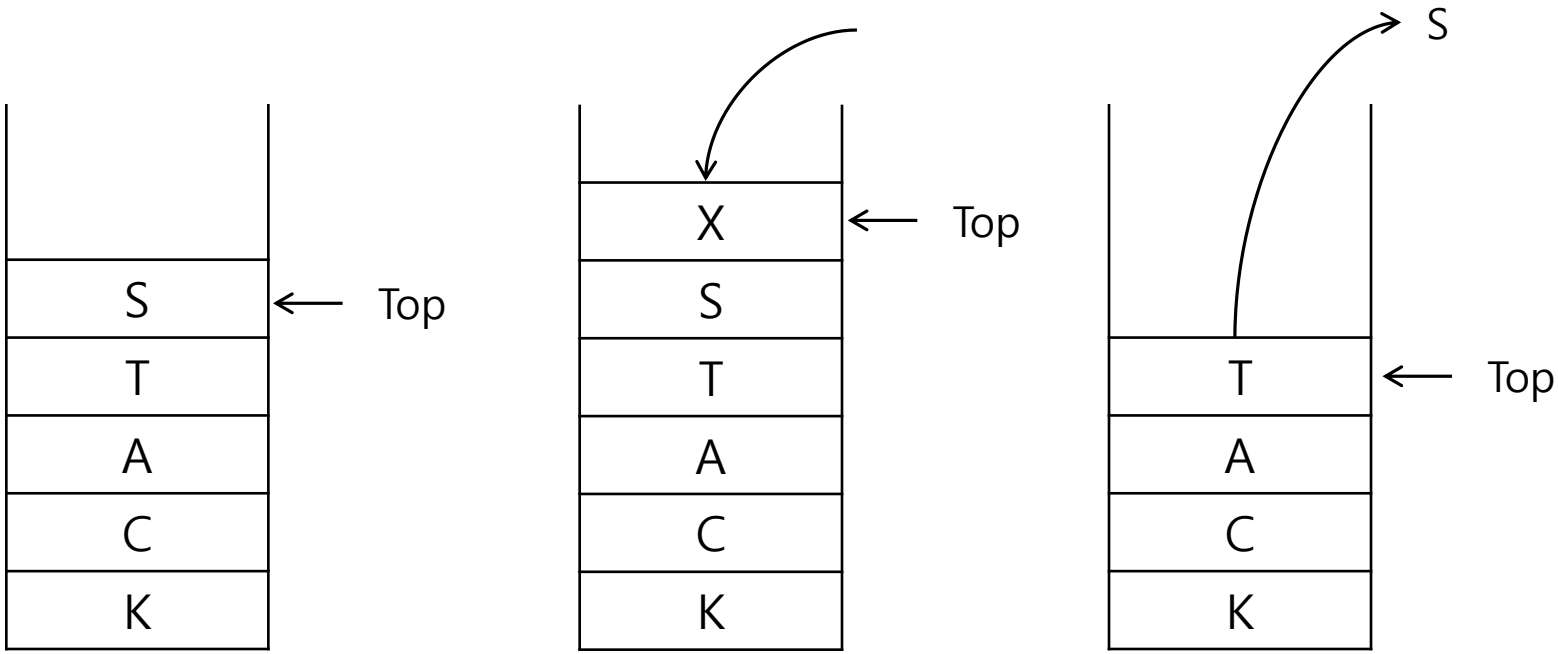
➔ **Linked list makes possible data structure to be flexible**

Introduction (2/2)

▪ Basic concept of stack (LIFO)

- **Two main functions : "push" and "pop"**

  - We need to track the position of "**top**" to manage stack !



[Stack] [Stack (push)] [Stack (pop)]

Array-based Stack (1/3)

- **Control of top index for handling the array**
  - **Easy to implement, however, the size of stack is fixed (static)**

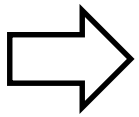
```
#define MAX 10
```
→ **Size of stack is fixed !**

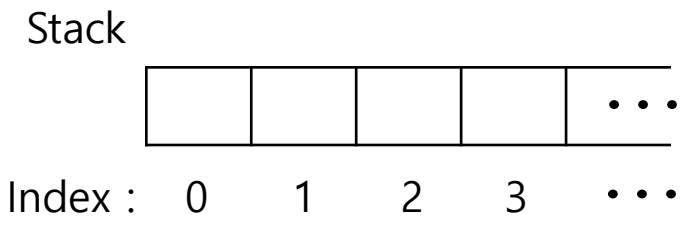
```
int stack[MAX];
int top;
```
→ **Index for the top of stack**

  - **Initialization of your stack**

- By setting the index of top as "-1", the stack can be initialized (why ?)

```
void init_stack(void)
{
   top = -1;
}
```





Array-based Stack (2/3)

- **Push and pop**
  - **Consider about stack "overflow" and "underflow"**

```
int push(int t)
{
    if(top >= MAX-1)
    {
        printf("Stack overflow !!!₩n");
        return -1;
    }

    stack[++top] = t;
    return t;
}
```

➔ **Explain the procedure (push)**

```
int pop()
{
    if(top < 0)
    {
        printf("Stack underflow !!!₩n");
        return -1;
    }

    return stack[top--];
}
```

➔ **Explain the procedure (pop)**

Array-based Stack (3/3)

- **Implementation of array-based stack**
  - **Definition of the node**

```
#define MAX 10

int stack[MAX];
int top;

void main()
{
    int k;
    init_stack();

    Make your code

}
```

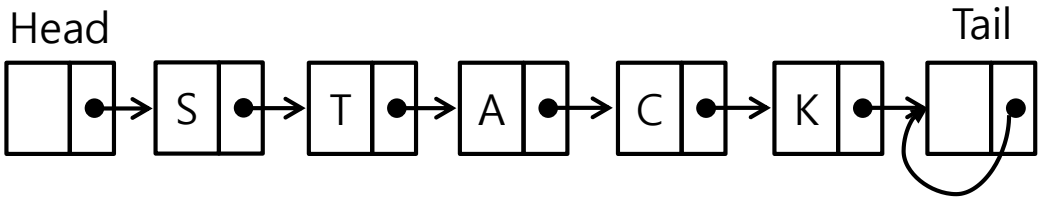
To do list :

1. Push 3, 6, 9, 1, 6, 3 and print top→bottom
2. Conduct pop and print top→bottom
3. Push 4, 8, 7, 2, 0 (what happens ?)
4. Initialize your stack
5. Conduct pop (what happens ?)

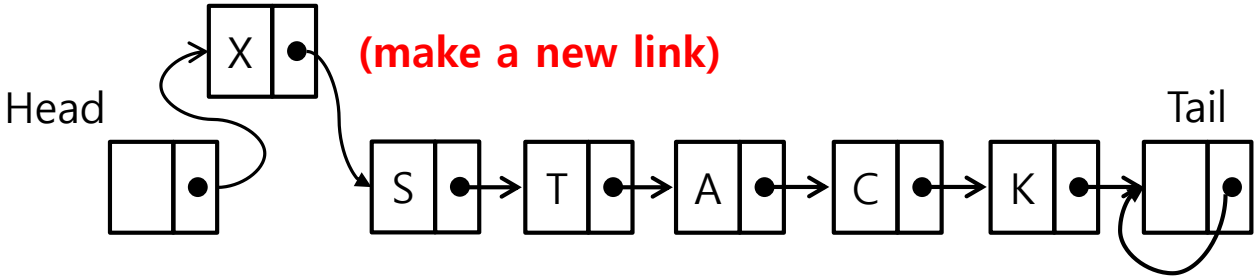
Linked List-based Stack (1/5)

- **Basic concept of linked list-based stack**
  - **Push : insert a new node / Pop : delete a node**

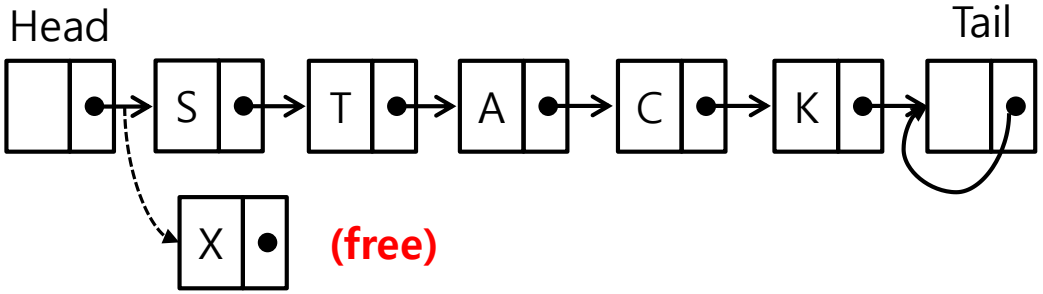
LL-based stack :



Push "X" :



Pop "X" :



Linked list-based Stack (2/5)

- **Flexible memory management !**
  - **Size of stack can be changed (reduction and increase)**
    - One entrance and one exit ➔ **simply linked list**

*** Structure definition**

```
typedef struct _node
{
    int key;
    struct _node *next;
}node;

(globally defined)
node *head, *tail;
```

*** Stack initialization**

```
void init_stack()
{
    head = (node *)calloc(1, sizeof(node));
    tail = (node *)calloc(1, sizeof(node));
    head->next = tail;
    tail->next = tail;
}
```

※ It is just same as that of simple liked list !

Linked list-based Stack (3/5)

- **Push and pop**
  - **Link operation (insert and delete)**

```
int push(int k)
{
    node *t;
    if((t = (node*)malloc
        (sizeof(node))) == NULL)
    {
        printf("Out of memory !₩n);
        return -1;
     }

    t->key = k;
    t->next = [                    ]
    head->next = [                    ]

    return k;
}
```

```
int pop()
{
    node *t;
    int k;
    if(head->next == tail)
    {
        printf("Stack underflow !₩n");
        return -1;
    }
    t = head->next;
    k = t->key;
    head->next = [                    ]
    free(t);

    return k;
}
```

Linked list-based Stack (4/5)

- **Clear your stack**
  - **Free all the memory spaces using pointers**

```
void clear()
{
    node *t, *s;
    t = head->next;

    while(t != tail)
    {
        s = t;
        t = t->next;
        free(s);
    }

    head->next = tail;
}
```

Why two pointers used ?

Draw the corresponding procedure

```
void print_stack()
{
    node *t;
    t = head->next;
    while(t != tail) {
        printf("%-6d", t->key);
        t = t->next;
    }
}
```

※ All the procedures are similar to those of simple liked list

Linked list-based Stack (5/5)

KU KONKUK UNIVERSITY

▪ Implementation of linked list-based stack

- **Definition of the node**

```
node *head, *tail;

void main()
{
    int k;
    init_stack();


    Make your code


}
```

To do list :

1. Push 3, 6, 9, 1, 6, 3 and print top→bottom
2. Conduct pop and print top→bottom
3. Push 4, 8, 7, 2, 0 (what happens ?)
4. Initialize your stack
5. Conduct pop (what happens ?)

☀ **What is the difference with array-based stack ?**

KU KONKUK UNIVERSITY

▪ From infix notation to postfix notation

- **We are familiar with using infix notation A*((B+C)/(D-E))**

Example : Convert (A+(B*C)) to the postfix notation

Pseudo code (for example) :

1. Ignore "(" and go further
2. Print the operand "A"
3. Push the operator "+"
4. When you meet ")", pop and print out

Display	Stack (top→ bottom)
A	

Display	Stack (top→ bottom)
A	+

Display	Stack (top→ bottom)
AB	+

Display	Stack (top→ bottom)
AB	*+

Estimate the final result !

Summary

- **Applications of linked lists**
  - **Stack and Queue (array-based vs. linked list-based)**
- **Think about the applications for stack and queue**
  - **Keyboard buffer, mouse operation, …**